

Multi-Source Candidate Data Transformer — Technical Design

Eightfold Engineering Intern Assignment (Jul–Dec 2026)

Pipeline

detect → **extract** → **normalize** → **merge** → **confidence** → **project** → **validate**. Source type is inferred from each input's path/URL (.csv, .json, .pdf/.docx, .txt, github.com/..., linkedin.com/in/...), with an explicit override available. Each source has its own extractor that pulls loose, source-shaped fields out (never raising — a malformed input degrades to zero records). Those loose fields are mapped + normalized into a *partial* canonical profile per source record. Partial profiles are grouped into one record per real candidate and conflict-resolved into a merged canonical profile with per-field and overall confidence. Finally, a projection layer reshapes the merged profile into either the full default schema or a runtime JSON config — same engine, no code changes.

Canonical Schema & Normalized Formats

Schema fields: candidate_id, full_name, emails[], phones[], location{city, region, country}, links[], headline, years_experience, skills[{name, confidence, source[]}], experience[{company, title, start, end, summary}], education[{institution, degree, field, end_year}], provenance[{field, source, method}], overall_confidence. **Phones** → best-effort E.164 (no external dep; assumes +1 for bare 10-digit US numbers, rejects anything outside 8–15 digits). **Country** → ISO-3166 alpha-2 via a lookup table over common spellings; **location** parsed from loose "City, ST" / "City, Region, Country" free text. **Dates** → YYYY-MM, accepting "Mar 2021", "3/2021", bare years, and "present"/"current" → null (ongoing). **Skills** → lowercased + a synonym table (js/es6→javascript, k8s→kubernetes, etc.).

Merge & Conflict-Resolution Policy

Match keys: normalized email first, normalized full name as fallback, joined via union-find so a record carrying only a name still links to a record that also has an email for the same person. **Scalar fields** (name, headline, location, years_experience): highest-reliability source wins, via a static prior — recruiter_csv 0.95, ats_json 0.90, github_api 0.80, linkedin_profile 0.65, resume_file 0.60, recruiter_notes 0.50 (recruiter/ATS-owned data trusted over self-reported free text). **List fields** (emails, phones, links, skills, experience, education): unioned and deduped, never overwritten — a candidate can legitimately have two emails or two jobs.

Confidence Assignment

Per-field confidence = max source reliability among contributing sources, +0.08 per additional independent corroborating source, capped at 0.99 (never claimed-certain). overall_confidence is a weighted average across populated fields, weighting identity fields (name, email) above peripheral ones (education).

Runtime-Configurable Output (Required Twist)

A JSON config controls projection with zero code changes: select a subset of fields; rename/remap via a *from* path into the canonical profile (dotted for nesting, [0] for list index, [].field for a list comprehension, e.g. skills[0].name); set a per-field normalization override independent of canonical normalization (E.164, canonical skill name, ISO2 country); toggle confidence/provenance inclusion; and choose an on_missing policy for required fields: **null** (keep key, value null), **omit** (drop the key), or **error** (exclude that candidate from output, log why). validate.py then checks the projected record against the exact config that produced it, so the schema actually delivered can never silently drift from the schema promised.

```
{ "fields": [
  {"path": "primary_email", "from": "emails[0]", "required": true},
  {"path": "phone", "from": "phones[0]", "normalize": "E.164"},
  {"path": "skills", "from": "skills[].name", "normalize": "canonical"} ],
  "include_confidence": true, "on_missing": "null" }
```

Edge Cases Handled / Left Out Under Time Pressure

- Empty/blank CSV rows and malformed JSON — skipped, never crash the run.
- A field present in one source, absent in another — merge unions what exists rather than nulling the field.
- Conflicting values for the same field across sources — resolved by source reliability, not last-write-wins.
- Garbage-typed values (e.g. years_experience as the string "garbage-not-a-number") — type-checked out, treated as absent rather than propagated.
- Candidate identifiable only by email, with full_name null in every source — merges correctly via the email key.
- **Left out:** fuzzy/phonetic name matching — exact-normalized-string only. A same-named record with no shared email anywhere in its group will NOT merge with its true match (demonstrated deliberately in the sample data: a 'Marcus Lee' from CSV/ATS with no email stays separate from a 'Marcus Lee' in recruiter notes that has one). Also out of scope: ML-based resume section parsing, live LinkedIn API (no public partner-free endpoint exists, so that source is vendor-JSON-in by design), and a web UI (CLI only, per the assignment's stated priority).